

Pointer Members and Deferred-Length Strings

Rohit Goswami · 2026-03-27 · python · fortran · numpy · f2py · ramblings

A pointer is a promise about data you do not own. A deferred-length string is a promise about storage you have not yet allocated. Wrapping either for Python means honoring contracts that C cannot express.

Background

Last time covered allocatable array members: manual Fortran wrappers, the C layer with NumPy, and how f2py automates it all. Two more member types are needed to round out the wrapping story: pointer components and deferred-length character strings.

Pointer components (F2018 7.5.4.6)[^{^fn:1}] do not own their data. They reference existing storage via pointer association (F2018 10.2.2). Deferred-length allocatable characters (F2018 7.4.4.2 paragraph 3) use the colon syntax `character(:)`, `allocatable` for strings whose length is determined at runtime.

As before, we build each layer by hand first, then show how f2py generates the similar but automated code. F2PY code here corresponds to the [post/f2py-pointers-and-strings](#) tag.

Pointer members

The Fortran type

A `DataView` holds a label and a pointer to a shared array. The shared array has the `target` attribute (F2018 8.5.17):

```
module pointer_member_mod
  use iso_c_binding
  implicit none
  double precision, target :: shared_data(5)

  type :: DataView
    integer :: label
    double precision, pointer :: view(:) => null()
  end type DataView

contains
  subroutine init_shared_data()
    shared_data = [10.0d0, 20.0d0, 30.0d0, 40.0d0, 50.0d0]
  end subroutine

  subroutine attach_view(dv)
    type(DataView), intent(inout) :: dv
    call init_shared_data()
    dv%view => shared_data
  end subroutine

  subroutine detach_view(dv)
    type(DataView), intent(inout) :: dv
    nullify(dv%view)
  end subroutine
end module
```

The `=> null()` default initialization means the pointer starts disassociated rather than undefined. This is good practice and required a crackfortran fix [^fn:2].

How pointers differ from allocatables

Allocatable members own their storage. Deallocating frees the memory. Pointer members reference storage owned by something else. Nullifying a pointer does not free the target; it just breaks the association.

This drives the wrapping strategy: allocatable members get both getters and setters. Pointer members get only a getter. A setter would require managing pointer association and target lifetimes across the Fortran/C boundary, and getting it wrong produces dangling pointers with no runtime check[^fn:3].

Manual Fortran wrappers

The wrappers use `associated()` (F2018 16.9.16) instead of `allocated()` :

► Full pointer wrapper module (click to expand)

Compare with the allocatable wrappers from the previous post. The structure is identical: take a `c_ptr`, call `c_f_pointer`, do the operation. The only difference is `associated()` instead of `allocated()`, and there is no setter.

Testing from C

► C test (click to expand)